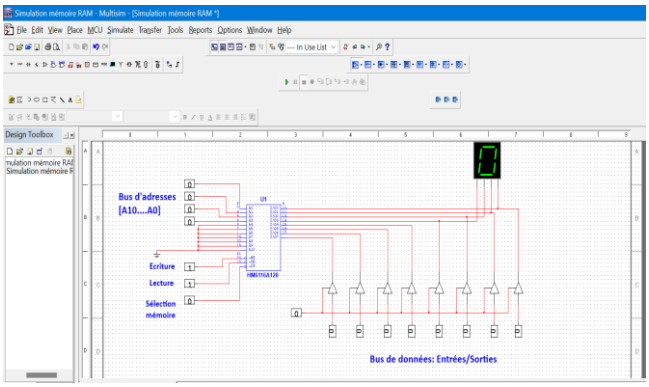
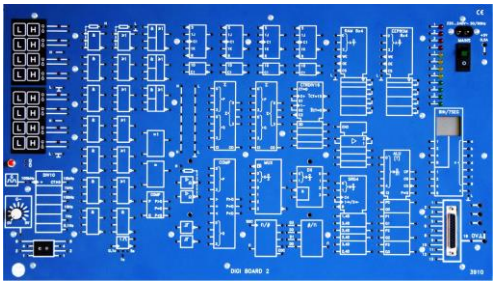
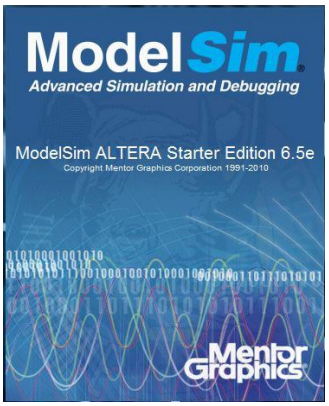
Université Ibn Zohr
Faculté des Sciences d'Agadir
Département de Physique

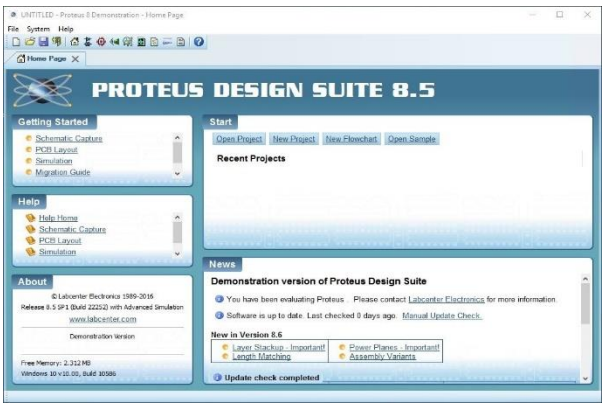
Filière : Électronique et Systèmes (ES)
Niveau : Semestre 5
Module : Electronique numérique 2





Travaux Pratiques :
Electronique numérique 2





Professeur responsable:
Noureddine MAOUHOUB

Année Universitaire : 2025-2026

Sommaire

Consignes aux étudiants……....3

TP N°1 : Circuits mémoires……..…….4

TP N°2 : Simulation des Circuits logiques programmables…….....…….…….…….11

TP N°3 : Simulation des circuits numériques par VHDL…….....18

Annexe……..24

Consignes importantes pour les étudiants

Pour le bon déroulement des travaux pratiques, les étudiants devront impérativement prendre en considération les consignes suivantes :

- Les étudiants sont tenus de respecter impérativement l'heure de début de chaque séance de TP. Tout retard sera comptabilisé par une absence et ne pourra en aucun cas être rattrapé.
- Les étudiants doivent obligatoirement préparer le sujet du TP avant l'arrivée à la séance et répondre aux questions théoriques (la révision du cours et du TD est nécessaire).
- Chaque étudiant doit obligatoirement respecter son groupe de TP, et doit avoir son propre polycopié (polycopié de l'année en cours et non pas celui de l'année précédente).
- Tout montage pratique ou de simulation doit être tout d'abord vérifié par le professeur.

TP N°1: Circuits mémoires

Objectifs :
A l'issue de ce travail pratique, les étudiants vont :

- Tester le fonctionnement des registres de mémorisation sur la maquette DIGI BOARD 2.
- Tester le fonctionnement des mémoires intégrés à la maquette DIGI BOARD 2.
- Simuler le fonctionnement des mémoires RAM en utilisant le logiciel MULTISIM.

I- Partie Pratique

II-1 Registre de mémorisation

Un registre de mémorisation permet de mémoriser un mot binaire de n bits. Les informations sont stockées par les bascules en utilisant un signal d'écriture W (Write) connecté à l'horloge de chaque bascule. Les informations sont lues sur les sorties des bascules en utilisant un signal de lecture R (Read).

Fonctionnement :

- **W=1 et R =0** : Chargement des données Di dans le registre (écriture), chaque bascule copie la valeur de l'entrée Di dans Qi.
- **W=0 et R = 1** : Lecture des données d'entrée.

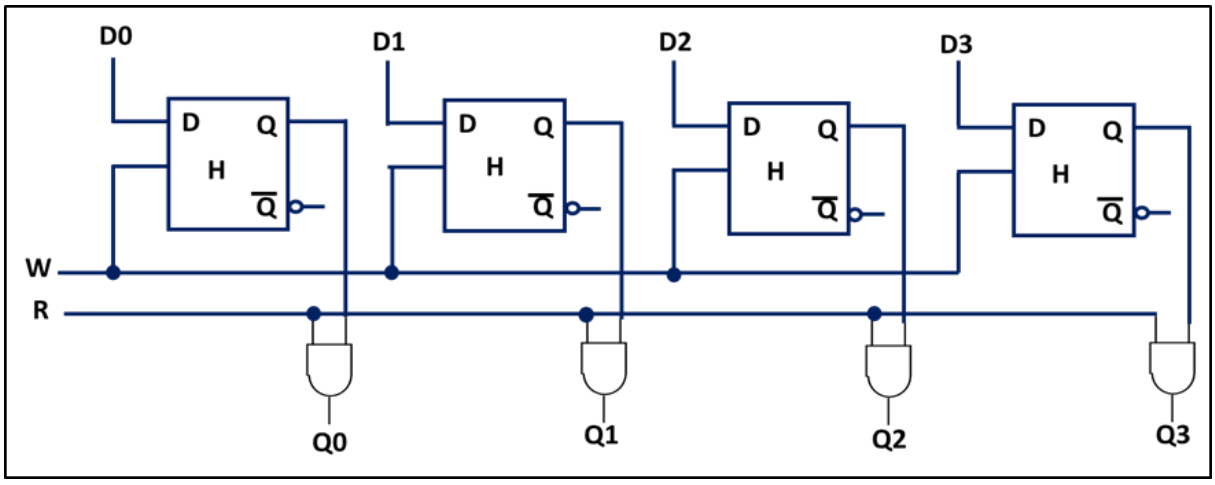


Figure 1.1 : Exemple de registre de mémorisation de 4 bits à base des bascules D actives sur niveau haut.

Manipulation

- Câbler le registre de mémorisation à quatre bits à base des bascules D intégrées à la maquette DIGIBOARD 2.
- Câbler les entrées W et R et les entrées D des bascules aux interrupteurs H et L.
- Câbles les sorties des bascules aux LED et compléter le tableau suivant :

Signaux de contrôle		Données d'entrées				Données de sortie			
W	**R**	**D0**	**D1**	**D2**	**D3**	**Q0**	**Q1**	**Q2**	**Q3**
1	0	1	0	1	0	….	….	….	….
0	1	1	0	1	0	….	….	….	….
1	0	1	1	0	0	….	….	….	….
0	1	1	1	0	0	….	….	….	….

II-1 Mémoire RAM intégrée à la maquette DIGIBOARD 2

Le circuit mémoire RAM intégrée à la maquette DIGIBOARD 2 est le suivant :

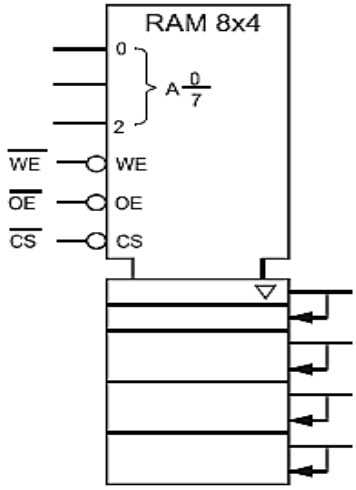


Figure 1.2 : Mémoire RAM intégrée à la maquette DIGIBOARD 2

II-1-1 Partie théorique

Répondre aux questions suivantes avant l'arrivée au TP

a- Capacité de cette mémoire en bits :……
b- Nombre de ligne dans le bus d'adresse :……
c- Nombre de mots adressées :……
d- Nombre de ligne dans le bus de données :…….
e- Donner les entrées de contrôle de cette mémoire et préciser leurs rôles :
……
……
……...

II-1-2 Partie pratique

II-1-2-1 Fonctionnement d'une mémoire RAM

- Câbler les lignes du bus d'adresse aux entrées logiques de la maquette.
- Câbler les entrées de contrôle aux entrées logiques de la maquette.

Pour le câblage des lignes du bus de données, vous allez procéder de la façon suivante :

- **Stockage de données (écriture)** : Câbler les lignes du bus de donnée aux entrées logiques de la maquette.
- **Lecture de données** : Câbler les lignes du bus de donnée au décodeur sept segments de la maquette.
- Mémoriser les informations du tableau ci-dessous.
- Vérifier les informations binaires stockées dans la mémoire (lecture)

NB : Pour mémoriser une information binaire dans la mémoire, utiliser la procédure suivante :

- Désactiver le mode écriture : $\overline{WE} = 1$
- Définir la valeur binaire de l'adresse de l'information à mémoriser.
- Définir la valeur binaire de l'information à mémoriser.
- Activer le mode écriture $\overline{WE} = 0$ pendant quelques secondes et puis désactiver ce mode d'écriture.

Adresses			**Ecriture** $\overline{CS}=0;\overline{WE}=0;\overline{OE}=1$				**Lecture** $\overline{CS}=0;\overline{WE}=1;\overline{OE}=0$			
A2	**A1**	**A0**	**D3**	**D2**	**D1**	**D0**	**D3**	**D2**	**D1**	**D0**
0	0	0	0	0	0	0	…	…	…	…
0	0	1	0	0	0	1	…	…	…	…
0	1	0	0	0	1	0	…	…	…	…
0	1	1	0	0	1	1	…	…	…	…
1	0	0	0	1	0	0	…	…	…	…
1	0	1	0	1	0	1	…	…	…	…
1	1	0	0	1	1	0	…	…	…	…
1	1	1	0	1	1	1	…	…	…	…

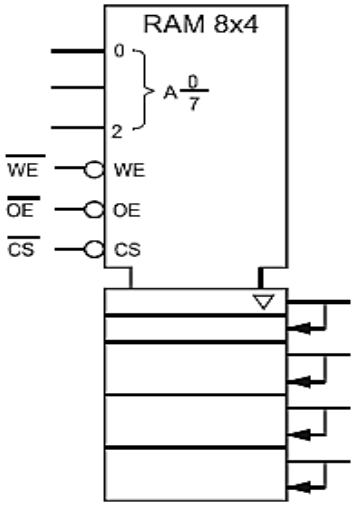
- Eteindre la maquette pendant quelques secondes et la mettre sous tension.
- Vérifier les données mémorisées dans la mémoire RAM. Que remarquez-vous ?.....
.....
.....

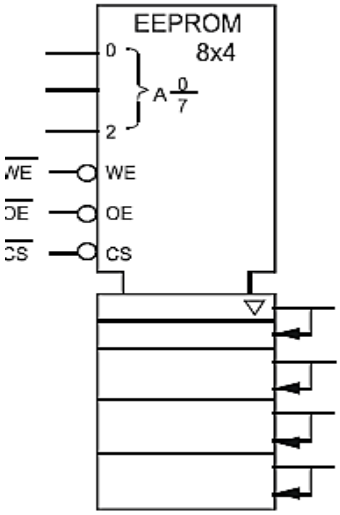
II-2 Assemblage des mémoires

II-2-1 Extension de nombre de ligne d'adresse

La mémoire EEPROM de la maquette DIGIBOARD 2 est une simulation, c'est-à-dire que si les composants ne sont pas alimentés, les informations restent stockées pendant environ 1 heure. Elle joue le même rôle que la RAM précédente.

- Câbler les deux mémoires afin d'avoir une mémoire 16×4 et d'augmenter le nombre de lignes d'adresse à quatre bits : A3,A2,A1,A0 dans ce cas le bit A3 est le bit de sélection (CS)



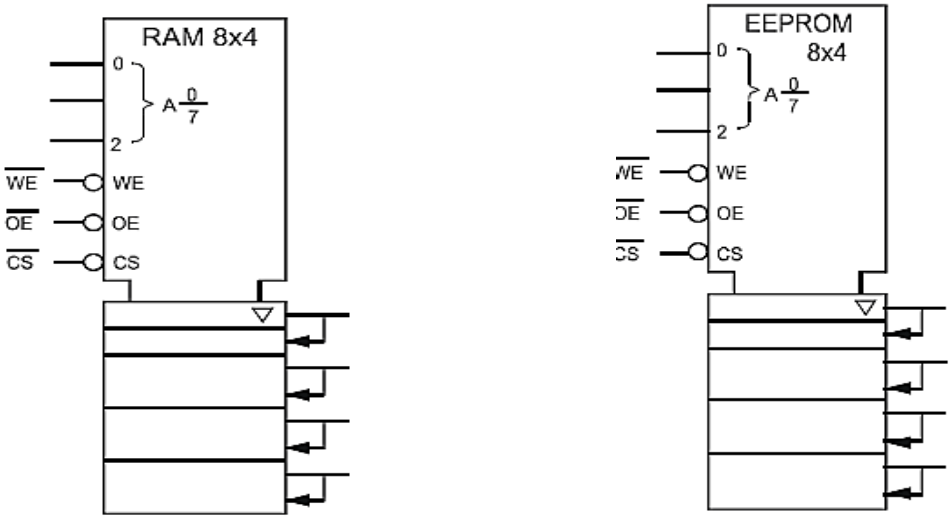


- Mémoriser les informations binaires dans le tableau ci-dessous (mode écriture).
- Vérifier les informations binaires stockées (mode lecture).

Adresses				**Ecriture**				**Lecture**			
A3	**A2**	**A1**	**A0**	**D3**	**D2**	**D1**	**D0**	**D3**	**D2**	**D1**	**D0**
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	…	…	…	…
0	**0**	**0**	**1**	**0**	**0**	**0**	**1**	…	…	…	…
0	**0**	**1**	**0**	**0**	**0**	**1**	**0**	…	…	…	…
0	**0**	**1**	**1**	**0**	**0**	**1**	**1**	…	…	…	…
0	**1**	**0**	**0**	**0**	**1**	**0**	**0**	…	…	…	…
0	**1**	**0**	**1**	**0**	**1**	**0**	**1**	…	…	…	…
0	**1**	**1**	**0**	**0**	**1**	**1**	**0**	…	…	…	…
0	**1**	**1**	**1**	**0**	**1**	**1**	**1**	…	…	…	…
1	**0**	**0**	**0**	**1**	**0**	**0**	**0**	…	…	…	…
1	**0**	**0**	**1**	**1**	**0**	**0**	**1**	…	…	…	…
1	**0**	**1**	**0**	**1**	**0**	**1**	**0**	…	…	…	…
1	**0**	**1**	**1**	**1**	**0**	**1**	**1**	…	…	…	…
1	**1**	**0**	**0**	**1**	**1**	**0**	**0**	…	…	…	…
1	**1**	**0**	**1**	**1**	**1**	**0**	**1**	…	…	…	…
1	**1**	**1**	**0**	**1**	**1**	**1**	**0**	…	…	…	…
1	**1**	**1**	**1**	**1**	**1**	**1**	**1**	…	…	…	…

II-2-2 Extension du bus de données

- Câbler les deux mémoires afin d'avoir une mémoire 8×8 et d'augmenter le bus de données à bits.



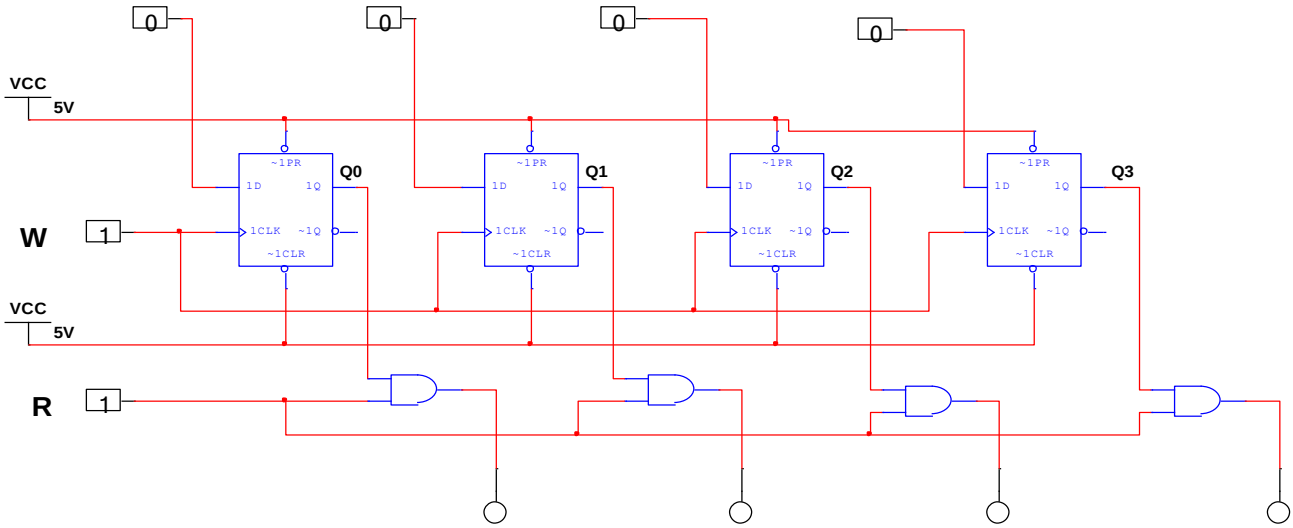
- Mémoriser les informations binaires dans le tableau ci-dessous (mode écriture).
- Vérifier les informations binaires stockées (mode lecture).

Adresses			**Ecriture**							
A2	**A1**	**A0**	**D7**	**D6**	**D5**	**D4**	**D3**	**D2**	**D1**	**D0**
0	**0**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**1**
0	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**1**	**1**
0	**1**	**0**	**0**	**1**	**0**	**1**	**0**	**1**	**0**	**1**
0	**1**	**1**	**0**	**1**	**1**	**1**	**0**	**1**	**1**	**1**
1	**0**	**0**	**1**	**0**	**0**	**1**	**1**	**0**	**0**	**1**
1	**0**	**1**	**1**	**0**	**1**	**1**	**1**	**0**	**1**	**1**
1	**1**	**0**	**1**	**1**	**0**	**1**	**1**	**1**	**0**	**1**
1	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**

III- Partie de simulation

III-1 Simulation d'un registre de mémorisation

- Réaliser le montage de simulation ci-dessous, sur MULTISIM d'un registre de mémorisation à quatre bits à base des bascules D.
- L'entrée d'horloge est la même entrée d'écriture W.
- Connecter l'entrée W, l'entrée R et les entrées D des quatre bascules à « INTERACTIVE_DIGITAL_CONSTANT » que vous allez la trouver dans le menu de « Place source ».
- Connecter les sorties des portes AND aux lampes « PROBE_DIG » que vous allez la trouver dans le menu de « Place indicator ».
- Tester le fonctionnement de ce registre de mémorisation en se basant sur le tableau de la page 5



III-2 Simulation d'une mémoire RAM 6116

III 2-1 Questions théoriques

Avant de réaliser la simulation sur MULTISIM, répondre aux questions suivantes avant l'arrivée au TP :

a- Capacité de la mémoire RAM 6116 en Kbits :……

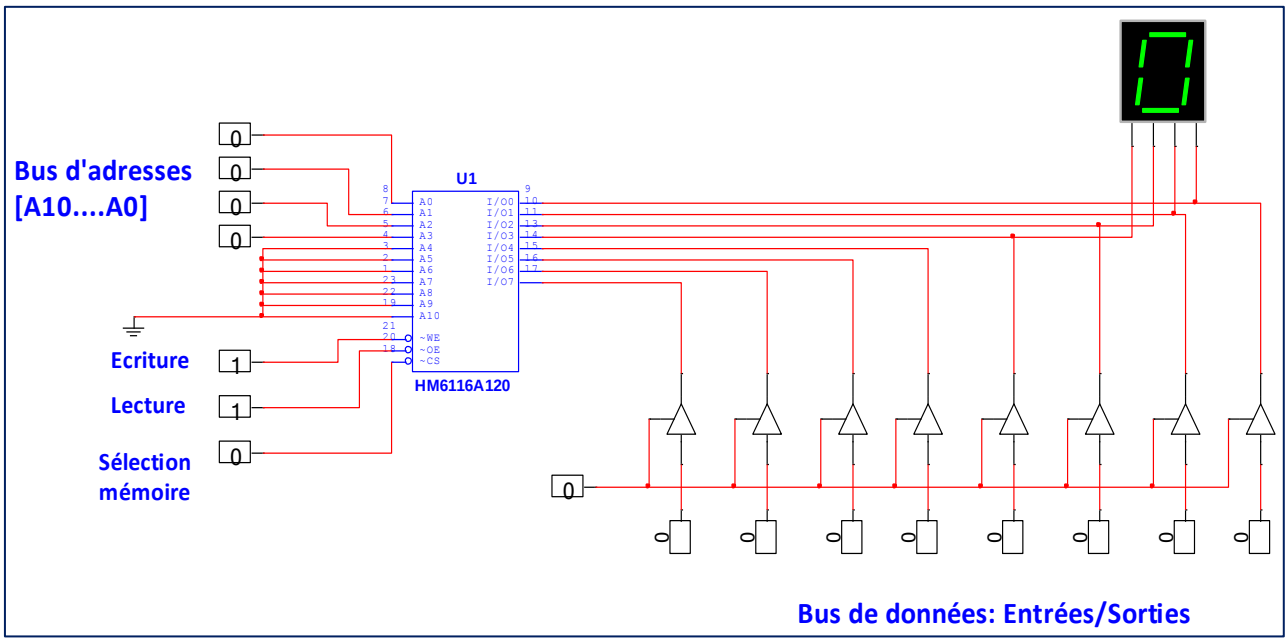
b- Nombre de ligne dans le bus d'adresse :……

c- Nombre de mots adressées :……

d- Nombre de ligne dans le bus de données :……

III 2-2 Simulation

- Réaliser le montage de simulation ci-dessous, sur MULTISIM de la mémoire RAM 6116.



- Afin de réduire le nombre d'adresses, nous allons utiliser seulement les quatre premières lignes d'adresse A0, A1, A2 et A3. Les autres lignes sont connectées à la masse.
- Connecter les quatre entrés des lignes d'adresse, les entrées de contrôle de sélection, de lecture et d'écriture aux « INTERACTIVE_DIGITAL_CONSTANT ».
- Connecter les Input/Output à un buffer à trois états « TRISTATE »
- Connecter les quatre sorties I/O0 à I/O3 à un afficheur hexadécimale « DCD_HEX_DIG_GREEN »
- Mémoriser les informations suivantes dans la mémoire RAM 6116 ($\overline{CS}=0;\overline{WE}=0;\overline{OE}=1$).

NB : Pour mémoriser une information binaire dans la mémoire, utiliser la procédure suivante :

- Désactiver le mode écriture : $\overline{WE}=1$
- Définir la valeur binaire de l'adresse de l'information à mémoriser.
- Définir la valeur binaire de l'information à mémoriser.
- Activer le mode écriture $\overline{WE}=0$ pendant quelques secondes et puis désactiver ce mode d'écriture.

Adresses					Ecriture				
A10…..A4	**A3**	**A2**	**A1**	**A0**	**I/O7..4**	**I/O 3**	**I/O 2**	**I/O 1**	**I/O0**
0	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**	**0**
0	**0**	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**1**
0	**0**	**0**	**1**	**0**	**0**	**0**	**0**	**1**	**0**
0	**0**	**0**	**1**	**1**	**0**	**0**	**0**	**1**	**1**
0	**0**	**1**	**0**	**0**	**0**	**0**	**1**	**0**	**0**
0	**0**	**1**	**0**	**1**	**0**	**0**	**1**	**0**	**1**
0	**0**	**1**	**1**	**0**	**0**	**0**	**1**	**1**	**0**
0	**0**	**1**	**1**	**1**	**0**	**0**	**1**	**1**	**1**
0	**1**	**0**	**0**	**0**	**0**	**1**	**0**	**0**	**0**
0	**1**	**0**	**0**	**1**	**0**	**1**	**0**	**0**	**1**
0	**1**	**0**	**1**	**0**	**0**	**1**	**0**	**1**	**0**
0	**1**	**0**	**1**	**1**	**0**	**1**	**0**	**1**	**1**
0	**1**	**1**	**0**	**0**	**0**	**1**	**1**	**0**	**0**
0	**1**	**1**	**0**	**1**	**0**	**1**	**1**	**0**	**1**
0	**1**	**1**	**1**	**0**	**0**	**1**	**1**	**1**	**0**
0	**1**	**1**	**1**	**1**	**0**	**1**	**1**	**1**	**1**

- Vérifier la lecture de ces informations binaires ($\overline{CS}=0;\overline{WE}=1;\overline{OE}=0$)
- Arrêter la simulation pendant quelques secondes.
- Vérifier les données mémorisées dans la mémoire RAM. Que remarquez-vous ?.....
.....
.....

TP N°2: Simulation des Circuits logiques programmables

Objectifs :

A l'issue de ce travail pratique, les étudiants vont :

- Simuler la programmation et l'implémentation des circuits logiques combinatoires et séquentielles sur des circuits logiques programmables en utilisant le logiciel de programmation WinCUPL et le logiciel de simulation ISIS PROTEUS.

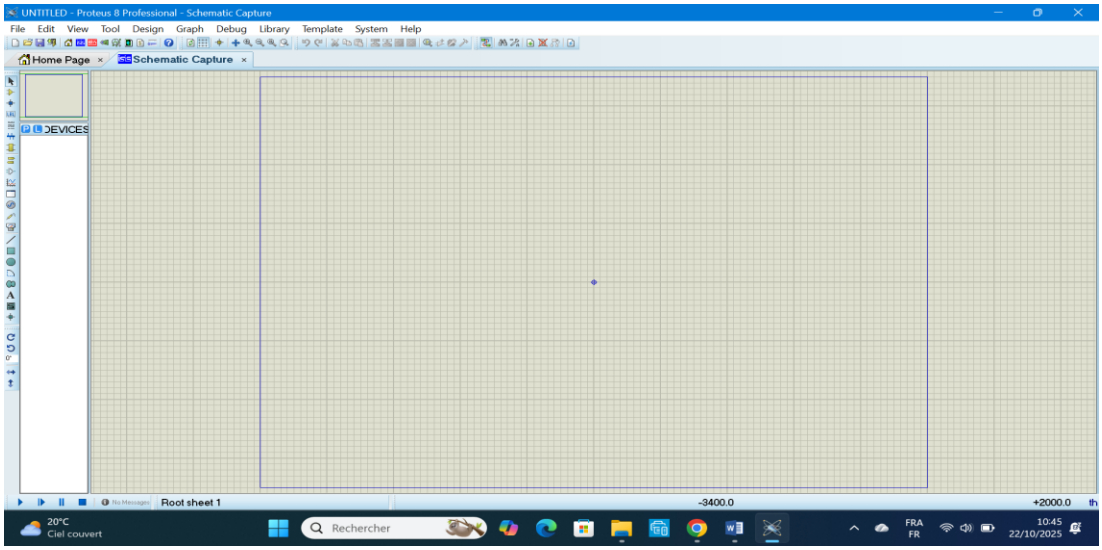
I- Présentation des logiciels ISIS PROTEUS et WinCUPL

I-1 ISIS PROTEUS

ISIS PROTEUS est un logiciel de conception et de simulation des circuits électroniques très utilisé en enseignement, en recherche et en ingénierie. Ce logiciel, développé par Labcenter Electronics, regroupe plusieurs modules, dont les plus connus sont :

- ISIS (Intelligent Schematic Input System) : pour dessiner des schémas électroniques et simuler le comportement des circuits.
- ARES (Advanced Routing and Editing Software) : pour concevoir les circuits imprimés (PCB).

L'interface graphique de ISIS est donné ci-dessous :

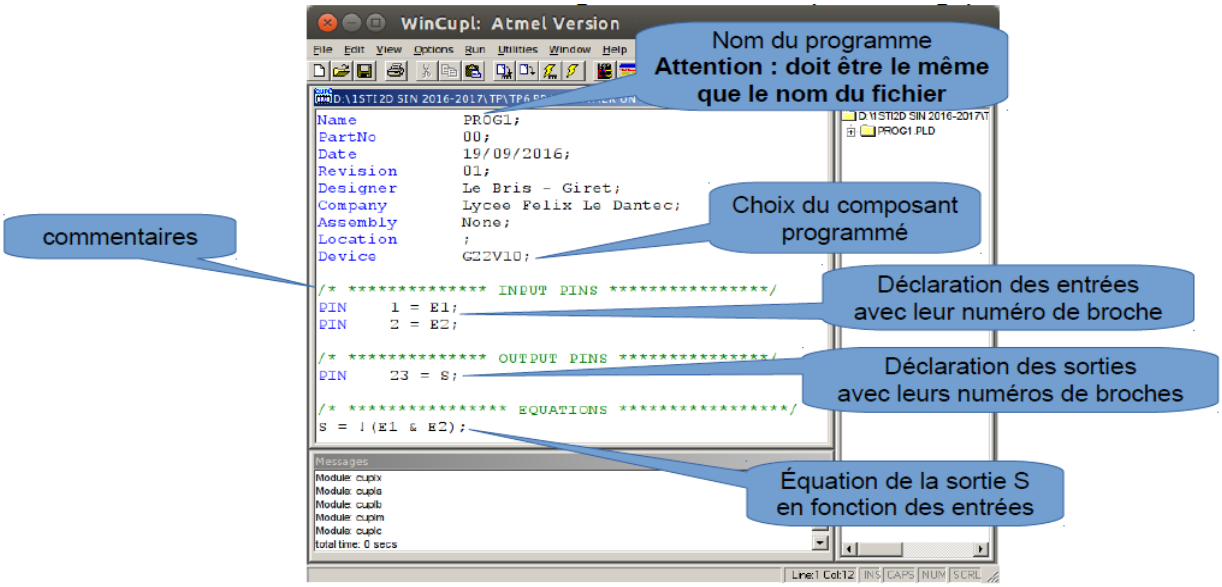


I-2 WinCUPL

WinCUPL est un logiciel gratuit développé par Microchip Technology (anciennement Atmel) pour la conception et la programmation des circuits logiques programmables, tels que les GAL, PAL et CPLD. Ce logiciel permet de :

- Ecrire des descriptions logiques (en langage CUPL), en utilisant un fichier. PLD. (Voir annexe).
- Compiler ces descriptions pour générer un fichier de programmation JEDEC(.jed). Le fichier JEDEC contient la liste des fusibles à détruire et à conserver et permet la programmation des circuits programmables grâce à un programmateur universel.
- Simuler ou tester les circuits logiques avant leur implémentation.

La figure ci-dessous montre l'interface graphique du logiciel.



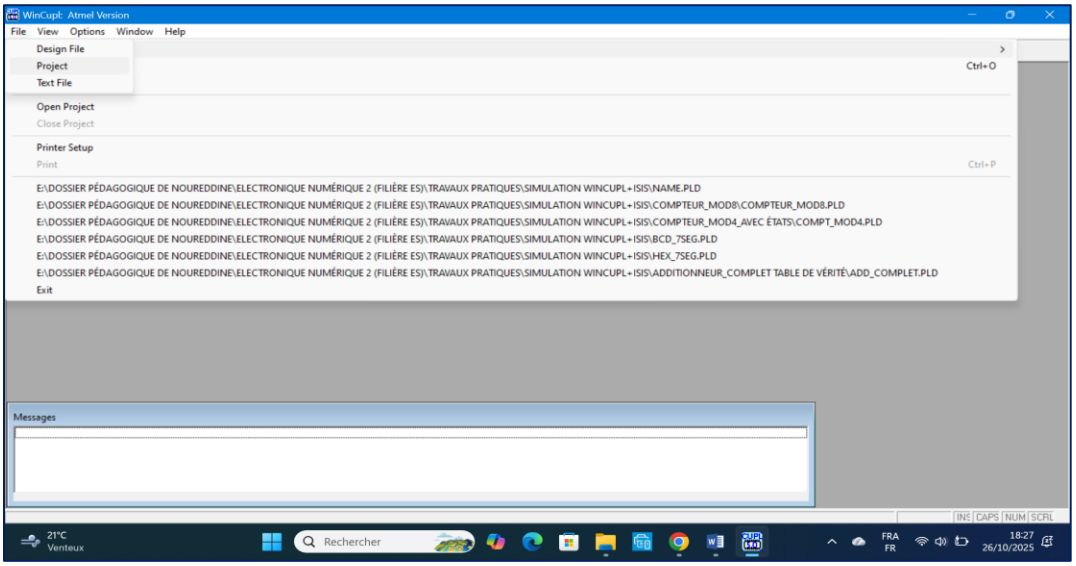
II- Programmation et simulation des circuits logiques combinatoires

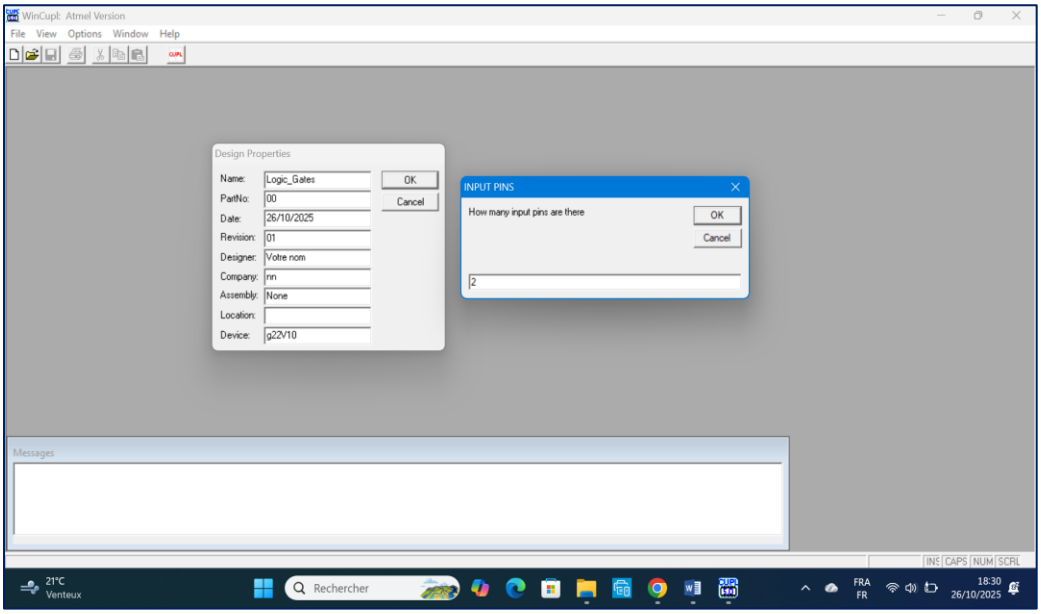
II-1 Programmation et simulation des portes logiques combinatoires

Dans cette partie, nous allons programmer les portes logiques de base (NOT, AND, OR, XOR, NAND et NOR) sur le logiciel WinCUPL et les simuler sous ISIS.

Etapes à suivre :

- Ouvrir le logiciel WinCUPL.
- Créer un nouveau projet nommé : Logic_Gates, puis choisir le circuit programmable utilisé (GAL 22V10). Ensuite, définir au départ le nombre d'entrées (INPUT PIN) (deux entrées) et de sortie OUTPUT PIN (six sorties).





- Utiliser votre nom dans le designer.
- Ecrire le programme ci-dessous.
- Compiler ce programme pour générer le fichier Logic_Gates.jed

```
Name     Logic_Gates ;
PartNo   00 ;
Date     26/07/2025 ;
Revision 01 ;
Designer Pr.MAOUHOUB ;
Company  nn ;
Assembly None ;
Location  ;
Device   G22V10 ;

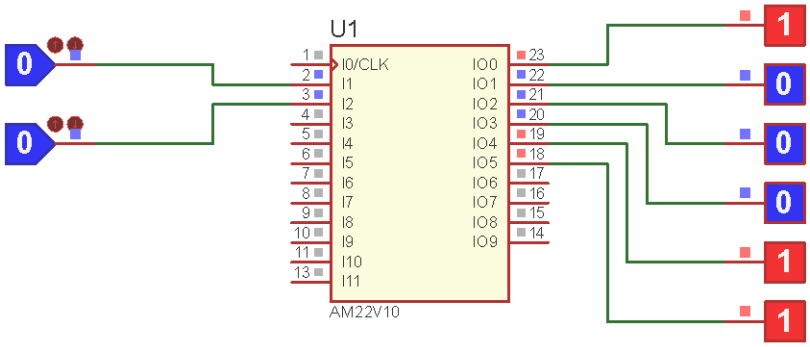
/* *************** INPUT PINS *********************/
PIN    2 =     A                    ;
PIN    3 =     B                    ;

/* *************** OUTPUT PINS *********************/
PIN  23 = S1                          ;
PIN  22 = S2                          ;
PIN  21 = S3                         ;
PIN  20 = S4                        ;
PIN  19 = S5                      ;
PIN  18 =  S6                     ;

/******* Equations***************/
S1 = !A;               /* Inverseur  */
S2 = A&B;              /* Porte AND  */
S3 = A#B;              /* Porte OR */
S4 = A$B;              /* Porte XOR */
S5 = !(A&B);           /* Porte NAND */
S6 = !(A#B);            /* Porte NOR */
```

- Ouvrir le logiciel ISIS PROTEUS.
- Réaliser le circuit de simulation suivant :
- Ouvrir le circuit AM22V10 à partir de « Devices » puis de « PLDs & FPGAs ».

- Connecter les entrées aux « LOGICSTATE » et les sorties aux « LOGICPROBE » qui se trouvent dans « Debugging Tools ».
- Double clic sur le circuit AM22V10, puis télécharger le fichier Logic_Gates.jed dans le circuit.
- Lancer la simulation est vérifier la table de vérité des portes logiques programmées.



II-2 Programmation et simulation d'un demi-additionneur

Dans cette partie, nous allons programmer et simuler le fonctionnement d'un additionneur complet sur le circuits programmable 22V10 en se basant sur les équations logiques et aussi sur la table de vérité.



- Rappeler la table de vérité d'un additionneur complet.
- Donner les équations logiques de cet additionneur.

a- Programmation par utilisation des équations logiques

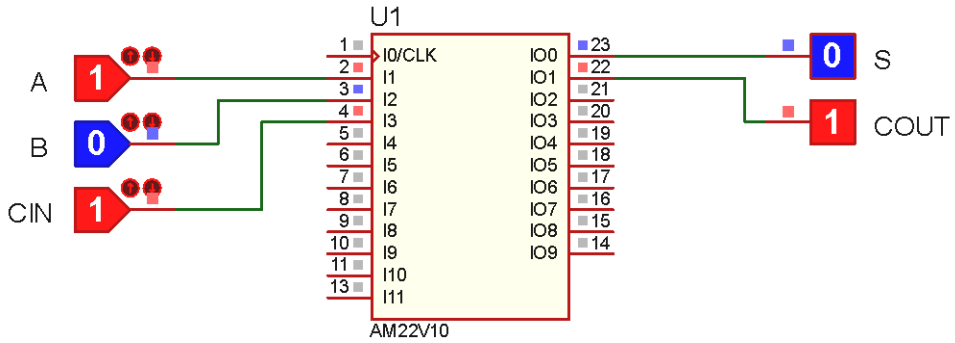
- Compléter le programme CUPL de l'additionneur complet en se basant sur les équations logiques des sorties.

```
Name     Additionneur_complet ;
PartNo   00 ;
Date     29/07/2025 ;
Revision 01 ;
Designer Pr.MAOUHOUB ;
Company  nn ;
Assembly None ;
Location  ;
Device   g22v10 ;

/* *************** INPUT PINS *********************/
PIN     2 = A                          ; /*                                */
PIN     3 = B                          ; /*                                */
PIN     4 = Cin                          ; /*                                  */

/* *************** OUTPUT PINS *********************/
PIN     23 = S                          ; /*                                 */
PIN     22 = Cout                          ; /*                                    */
.............................
..............................
```

- Compiler le programme et générer le fichier .jed, et simuler le fonctionnement de ce circuit sous ISIS PROTEUS.
- Vérifier la table de vérité de l'additionneur complet.



b- Programmation par utilisation de la table de vérité

Compléter le programme CUPL de l'additionneur complet en se basant sur sa table de vérité.
- Pour cela on définit un bus d'entrée et un bus de sortie à l'aide de la commande « Field »
- Pour créer une table de vérité on utilise la commande « Table »
- Compiler le programme et générer le fichier .jed, puis simuler son fonctionnement sur ISIS.
- Vérifier la table de vérité de l'additionneur complet.

```
Name     Additionneur_complet ;
PartNo   00 ;
Date     29/07/2025 ;
Revision 01 ;
Designer Pr.MAOUHOUB ;
Company  nn ;
Assembly None ;
Location  ;
Device   g22v10 ;

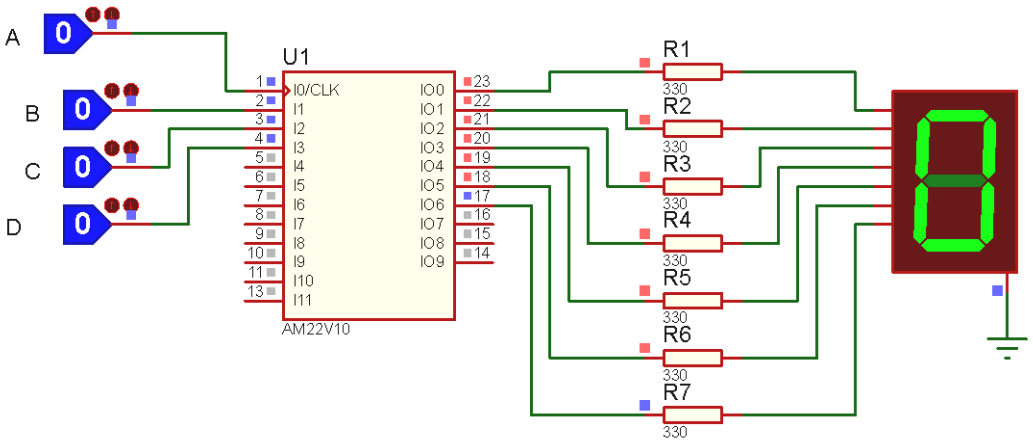
/* *************** INPUT PINS *********************/
PIN     2 = A                          ; /*                                */
PIN     3 = B                          ; /*                                */
PIN     4 = Cin                          ; /*                                  */

/* *************** OUTPUT PINS *********************/
PIN     23 = S                          ; /*                                 */
PIN     22 = Cout                          ; /*                                    */

field entree = [A,B,Cin];
field sortie = [S,Cout];
TABLE entree => sortie {
    'b'000 => 'b'00;
    'b'001 => 'b'10;
……….
……
……..
……
```

II-3 Programmation et simulation d'un décodeur BCD sept segments

On propose de programmer le décodeur BCD sept segment en utilisant le circuit logique programmable GAL 22V10, et de simuler son fonctionnement sur ISIS PROTEUS. La figure de simulation est montrée dans la figure ci-dessous :



L'afficheur sept segment à utiliser dans la simulation est : « 7SEG-COM-CAT-GRN » qui se trouve dans « Optoelectronics ».
La table de vérité de ce décodeur est donnée par :

Décimal	DCBA	a	b	c	d	e	f	g
0	0000	1	1	1	1	1	1	0
1	0001	0	1	1	0	0	0	0
2	0010	1	1	0	1	1	0	1
3	0011	1	1	1	1	0	0	1
4	0100	0	1	1	0	0	1	1
5	0101	1	0	1	1	0	1	1
6	0110	1	0	1	1	1	1	1
7	0111	1	1	1	0	0	0	0
8	1000	1	1	1	1	1	1	1
9	1001	1	1	1	1	0	1	1

- Donner le programme CUPL de ce décodeur BCD-7SEG en utilisant la table de vérité.
- Compiler ce programme et générer le fichier .jed
- Réaliser le montage de simulation sur ISIS et tester le fonctionnement du décodeur.

III- Programmation et simulation des circuits logiques séquentiels

III-1 Programmation et simulation d'un compteur synchrone modulo 8 à base de bascules D

- Rappeler les équations logiques des sortie Qi d'un compteur synchrone modulo 8 en fonction des entrées Di
- Compléter le programme CUPL ci-dessous.
- la commande Qi.D permet de définir la sortie Qi d'une bascule D et de la réinjecter dans la l'entrée de la bascule Di.

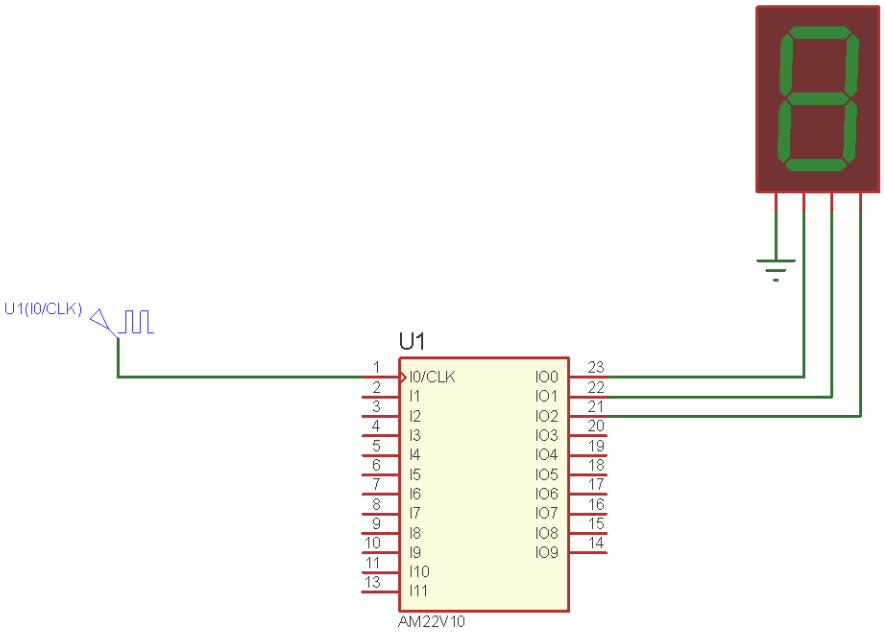
```
Name     compteur_mod8 ;
PartNo   00 ;
Date     28/07/2025 ;
Revision 01 ;
Designer Pr.MAOUHOUB ;
Company  nn ;
Assembly None ;
Location  ;
Device   g22v10 ;
```

```
/* *************** INPUT PINS *********************/
PIN     1 = CLK                          ; /*                                 */

/* *************** OUTPUT PINS *********************/
PIN     23 = Q2                         ; /*                                */
PIN     22 = Q1                          ; /*                                 */
PIN     21 = Q0                          ; /*                                 */


Q2.D =
Q1.D =
Q0.D =
```

- Compiler le programme pour générer le fichier .jed
- Réaliser sur ISIS le montage ci-dessous. L'horloge de synchronisation doit etre connecté à l'entrée d'horloge I0/CLK.
- Télécharger le fichier .jed dans le circuit GAL 22V10 et tester le fonctionnement du compteur.



TP N°3 : Simulation des circuits numériques par VHDL

Objectifs :

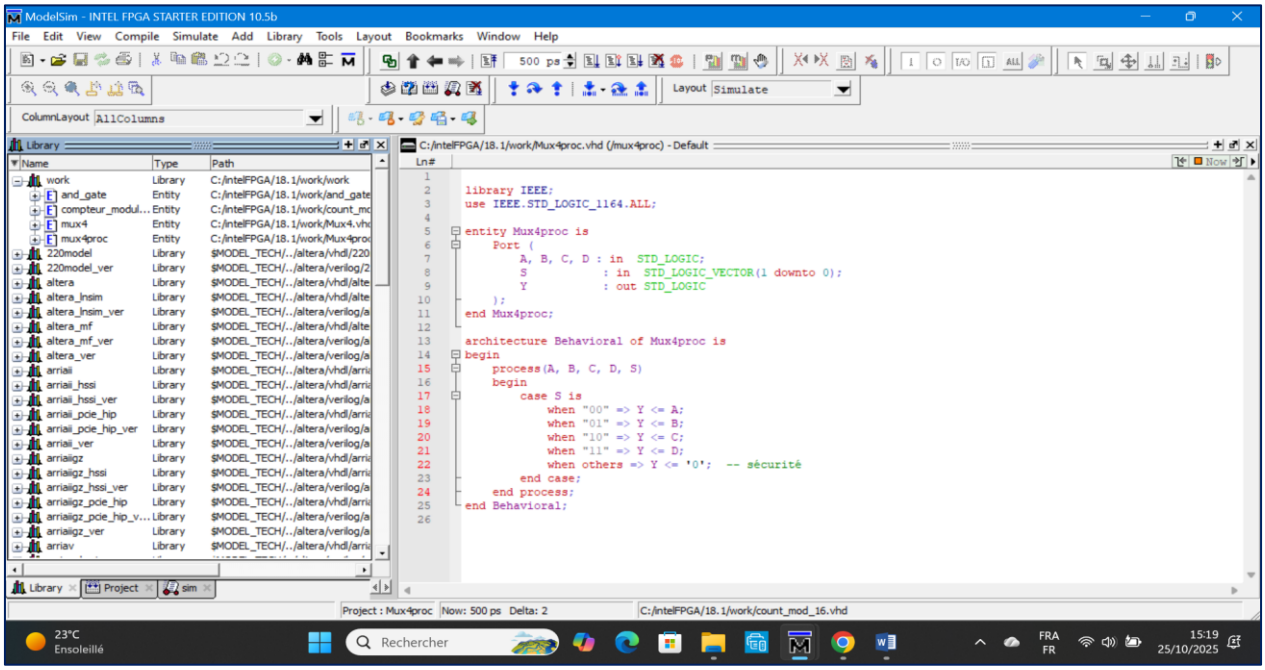
A l'issue de ce travail pratique, les étudiants vont :

- Concevoir par utilisation du langage de description matériel VHDL quelques circuits numériques de base combinatoires (portes logiques, additionneur complet, multiplexeurs, comparateur, décodeurs…) et séquentiels (registres et compteurs)
- Simuler le fonctionnement de ces circuit par utilisation du logiciel MODELSIM

I- Présentation du logiciel MODELSIM

Le logiciel ModelSim Intel FPGA Starter Edition est une version gratuite et limitée du simulateur de langage de description de matériel (HDL) ModelSim.

- Il est distribué par Intel avec sa suite logicielle Intel Quartus Prime Lite Edition pour la conception de circuits FPGA.
- Il permet aux développeurs de vérifier le bon fonctionnement et la temporisation de leurs conceptions de circuits logiques avant de les charger sur un circuit programmable FPGA ou CPLD.

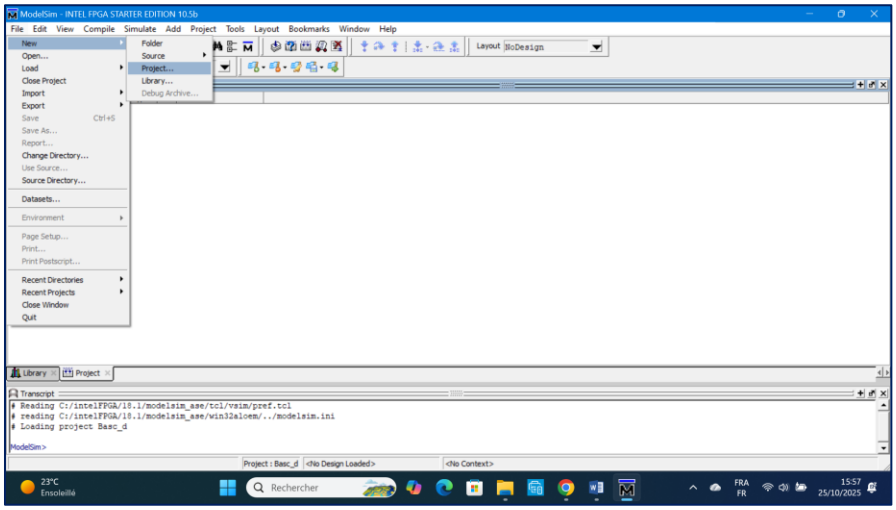


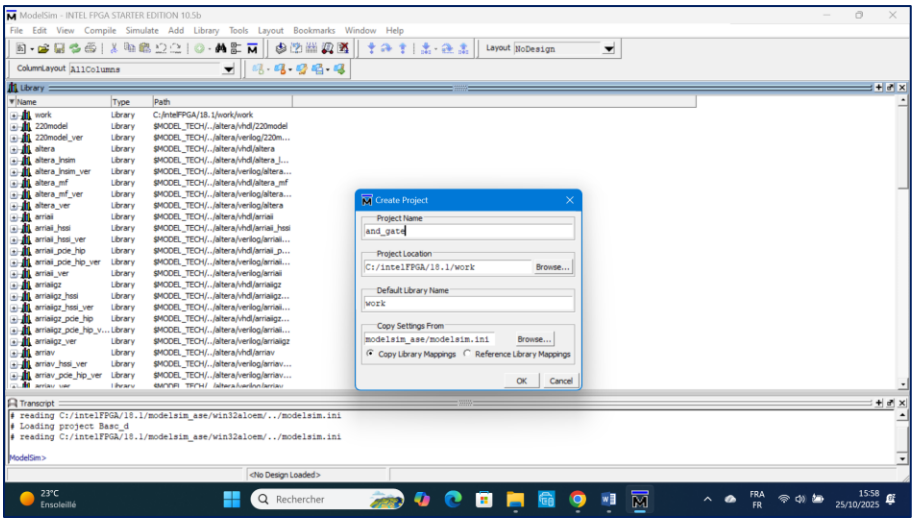
II- Description et simulation VHDL de quelques circuits logiques combinatoires

II-1 Description et simulation VHDL d'une porte logique AND

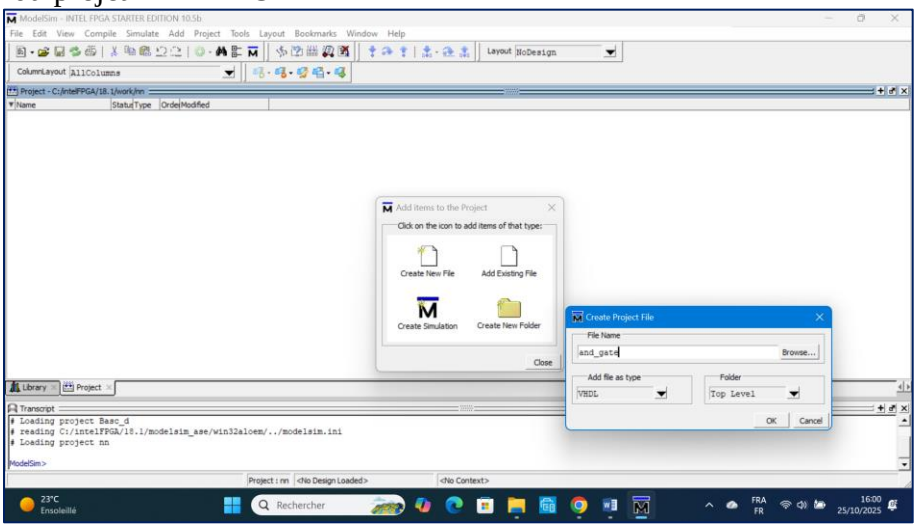
Pour réaliser la simulation d'un programme VHDL sous MODELSIM, on suit les étapes suivantes :

- Lancer le logiciel MODELSIM.
- Créer un nouveau projet nomé « and_gate » : Aller dans menu file-----> New------> Project, puis choisir par défaut l'emplacement d'enregistrement

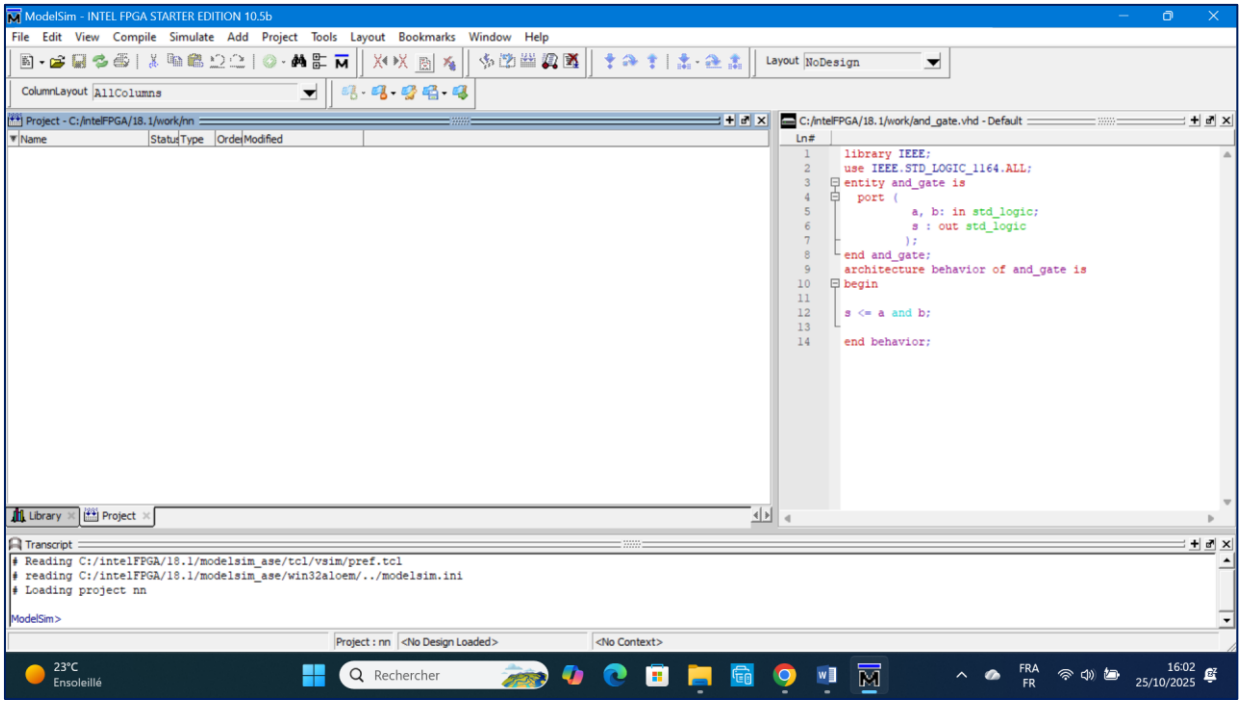




- Créer un nouveau fichier pour écrire la description VHDL : aller dans create new file----> choisir le même nom du projet---------> OK



- Double clic sur le fichier VHDL crée, puis écrire la description VHDL de la porte AND ci-dessous :



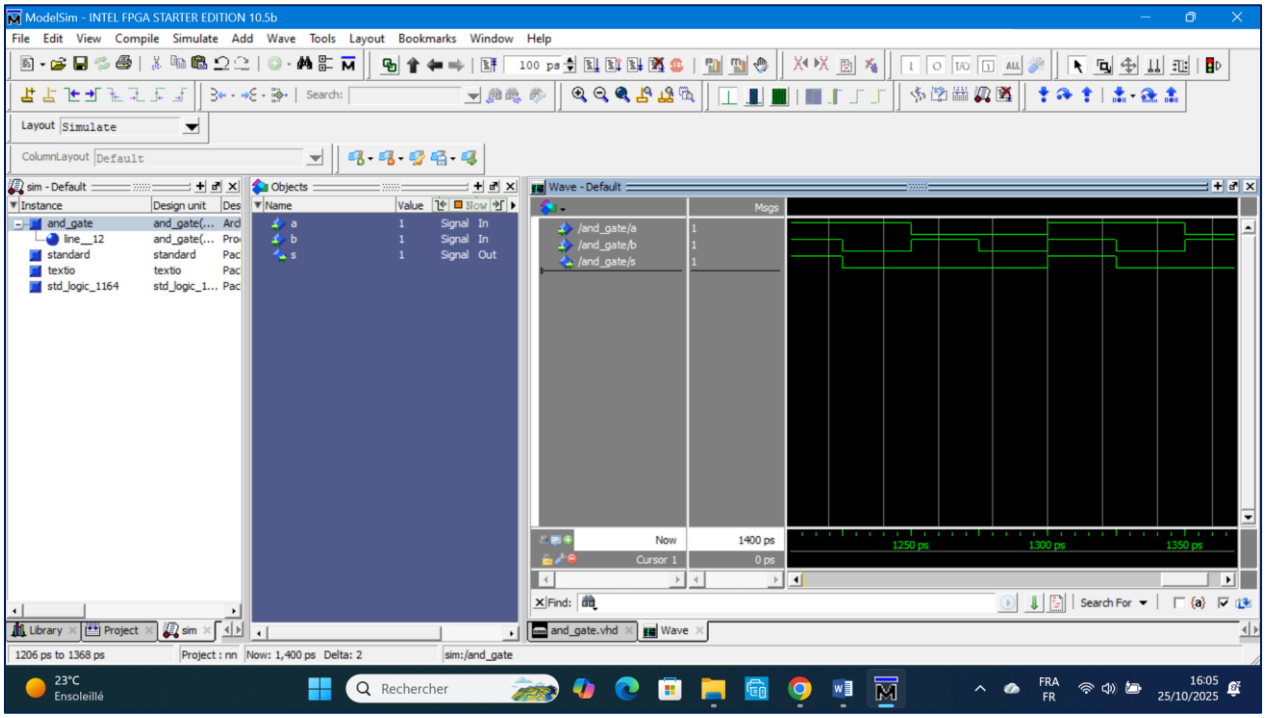

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity and_gate is
  port (
        a, b: in std_logic;
        s : out std_logic
       );
end and_gate;
architecture behavior of and_gate is
begin

s <= a and b;

end behavior;
```

- Compiler la description VHDL : utiliser compile all
- Vérifier qu'il n'y a pas d'erreurs.
- Pour simuler la description VHDL, on suit les étapes ci-dessous :
    - Aller dans le menu « Simulate »: cliquer sur « Start simulation », puis sélectionner dans « work » le nom du fichier à simuler.
    - Aller dans le menu « View » ouvrir la fenêtre « Wave » pour visualiser les signaux de simulation.

- Sélectionner les signaux du circuit est les ajouter dans la fentre « Wave » en utilisant un clic par le bouton droit de la sourie, puis « add wave ».
- Appliquer des signaux d'horloge aux entrées de fréquences différentes : clic droit sur le signal-----> clock-------> choisir la périodes du signal
- Démarrer la simulation « Run » et vérifier le fonctionnement du circuit.



II-2 Description et simulation VHDL d'un additionneur complet

- Rappeler la table de vérité d'un additionneur complet (voir TP 2)
- Proposer un programme VHDL pour décrire le fonctionnement de cet additionneur) en utilisant les équations logiques ou en se basant sur la table de vérité

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
entity add_complet is
  port (
        a,b,cin:……
        s,cout:……
       );
end add_complet;
architecture behavior of add_complet is
begin

s <= ……..;
cout <= ……..;

end behavior;
```

II-3 Description et simulation VHDL d'un multiplexeur quatre vers un

- Rappeler la table de vérité d'un multiplexeur quatre vers un
- Compléter le programmes ci-dessous pour décrire un multiplexeur quatre vers un

```
library IEEE;
use IEEE.STD_LOGIC_1164.all;
entity Mux4 is
port (
      A,B,C,D: in std_logic;
      S: in std_logic_vector (1 downto 0);
      Y: out std_logic
       );
end Mux4;
Architecture fct of Mux4 is
begin
Y <= A when (S = "00") else
    …….
…….
end fct;
```

III- Description et simulation VHDL de quelques circuits séquentiels

III-1 Description et simulation VHDL d'une bascule D à front montant

- Rappeler la table de vérité d'une bascule D à front montant.
- Simuler la description VHDL ci-dessous d'une bascule D à front montant.
- Pour la simulation utiliser par exemple un signal clock à l'entrée D de période inférieur à celle du signal d'horloge.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;

entity Basc_d is
   Port (
      D   : in  STD_LOGIC;    -- Entrée de données
      CLK : in  STD_LOGIC;    -- Horloge
      Q   : out STD_LOGIC     -- Sortie
   );
end Basc_d;

architecture Behavioral of Basc_d is
begin
   process(CLK)
   begin
      if rising_edge(CLK) then
         Q <= D;
      end if;
   end process;
end Behavioral;
```

III-2 Description et simulation VHDL d'un compteur modulo 16

- Simuler la description VHDL suivante qui décrit un compteur modulo 16 à front montant et avec remise à zéro (entrée reset).

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;
entity compteur_modulo16 is
   Port (
      clk   : in  STD_LOGIC;          -- Horloge
      reset : in  STD_LOGIC;          -- Reset asynchrone
      count : out STD_LOGIC_VECTOR(3 downto 0)  -- Sortie compteur 4 bits
   );
end compteur_modulo16;

architecture Behavioral of compteur_modulo16 is
   signal compteur : unsigned(3 downto 0) := (others => '0'); -- compteur interne
begin
   process(clk, reset)
   begin
      if reset = '1' then
         compteur <= (others => '0');  -- Réinitialisation
      elsif rising_edge(clk) then
         if compteur = "1111" then
            compteur <= (others => '0');  -- Retour à 0 après 15
         else
            compteur <= compteur + 1;
         end if;
      end if;
   end process;
   count <= std_logic_vector(compteur);  -- Conversion en std_logic_vector
end Behavioral;
```

Annexe

Langage CUPL

❖ **Généralités :**

- Le nom du programme doit être le même que le fichier. pld enregistré.
- Les instructions se terminent par un point-virgule.
- Les commentaires doivent être précédés par /* et terminés par */. Ils ne sont pas exécutables mais permettent de décrire les différentes entrées et sorties et les différentes instructions.

❖ **Les nombres :**

Les nombres dans CUPL peuvent être écrits dans différentes bases en respectant les symboles suivants (exemple):

Nombre	Base	Valeur en décimal
'b'0	binaire	0
'b'1010	binaire	10
'o'12	octale	10
'd'82	décimale	82
'h'F	héxadécimale	15

❖ **Déclarations**

DEVICE : Définir me circuit PLD utilisé. Par exemple un GAL 22V10

```
Device G22V10;
```

• **PIN** : Associer un nom à une broche du circuit.

```
PIN 1 = A ; /* signal A défini sur la broche 1 */
PIN [1..4] = [A0..3]; /* signaux A0, A1, A2 et A3 sur broches 1, 2, 3 et 4 */
```

• **FIELD** : Cette commande permet de déclarer un **bus** afin de réunir des signaux.

```
FIELD ENTREE = [A3..A0]; /* bus ENTREE = A3 A2 A1 A0 */
```

❖ **Programmation d'une table de vérité**

Pour écrire et programmer une table de vérité d'un circuit numérique et définir les combinaisons possibles, nous utilisons la commande TABLE qui associe les combinaisons possible d'un bus d'entrée au bus de sortie (on peut utiliser les différentes bases : binaire, hexadécimale…:

```
TABLE ENTREE => SORTIE {
'b'00 => 'b'1110;
'b'01 => 'b'1101;
```

```
'b'10 => 'b'1011;
'b'11 => 'b'0111;
}
```

❖ **Ecriture des équations logiques**

Les équations logiques dans un programme CUPL, permettent de décrire la fonction logique réalisée par le circuit PLD. On utilise la description suivante pour écrire ces équations :

Nom de la sortie = expression booléenne des entrées;

• Écriture des opérations logiques

Opérateur	Exemple	Description	Précédence
!	!A	NOT	1
&	A&B	AND	2
#	A#B	OR	3
$	A$B	XOR	4

• Écriture des opérations arithmétiques

Opérateur	Exemple	Description	Précédence
**	4**	Exposant	1
*	3*5	Multiplication	2
/	4/2	Division	2
%	7%3	Modulo	2
+	3+5	Addition	3
-	5-2	Soustraction	3

Liens de téléchargement des logiciels

❖ WinCUPL

Pour plus de détail, vous pouvez télécharger le logiciel WinCUP et son manuel de ce logiciel, à partir du lien ci-dessous :

https://www.microchip.com/en-us/development-tool/wincupl

❖ ModelSim Intel FPGA starter

Pour télécharger le logiciel ModelSim Intel FPGA starter à partir du lien ci-dessous :

https://www.intel.com/content/www/us/en/software-kit/750368/modelsim-intel-fpgas-standard-edition-software-version-18-1.html